



Unidad II – Estilos y Patrones

Estilos Arquitectónicos

Arquitectura de Sistemas

Lic. Marcelo Palma

mpalma@uch.edu.ar
marcelo.palma.a@gmail.com

Recordemos que arquitectura es...



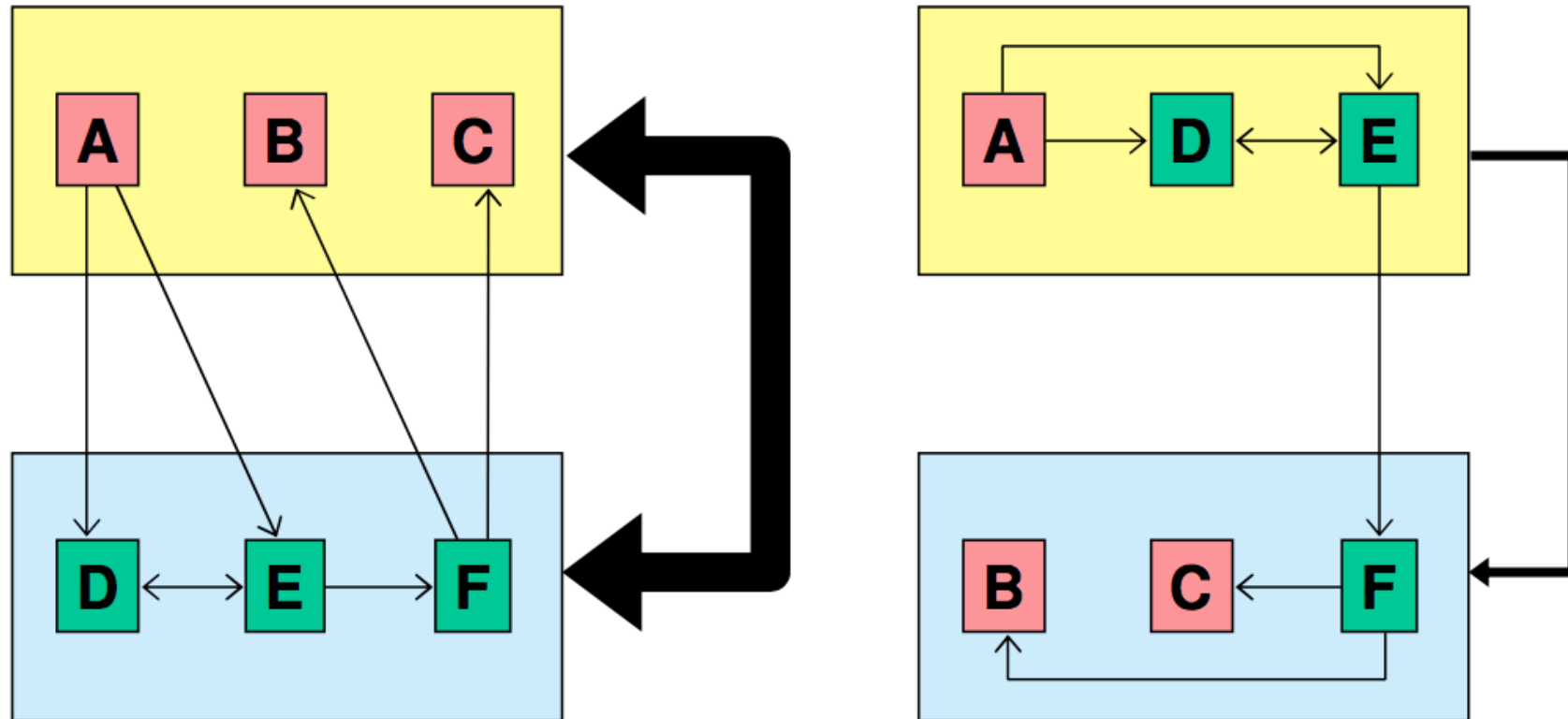
- Vista estructural de alto nivel
- Define estilo o combinación de estilos para una solución
- Se concentra en requisitos no funcionales (RNF)
 - Los requisitos funcionales se satisfacen mediante modelado y diseño de aplicación
- Esencial para el éxito o fracaso del desarrollos de software



Corrientes principales

- Arquitectura estructural – SEI – Carnegie Mellon
 - Garlan, Shaw, Clements
 - Variantes con modelos de datos (Medvidovic), radicales, formales (Moriconi-SRI), etc
- Arquitectura como etapa de la ingeniería de software orientada a objetos
 - James Rumbaugh, Grady Booch, Ivar Jacobson (“los 3...”), Craig Larman...
- Arquitectura basada en patrones – SEI
 - Redefinición de estilos como patrones POSA
 - Microsoft Patterns & Practices
- Arquitectura procesual y metodologías
 - Kazman, Bass (SEI)
 - Variantes de arquitectura basada en escenarios

Cómo lograr una buena descomposición modular?

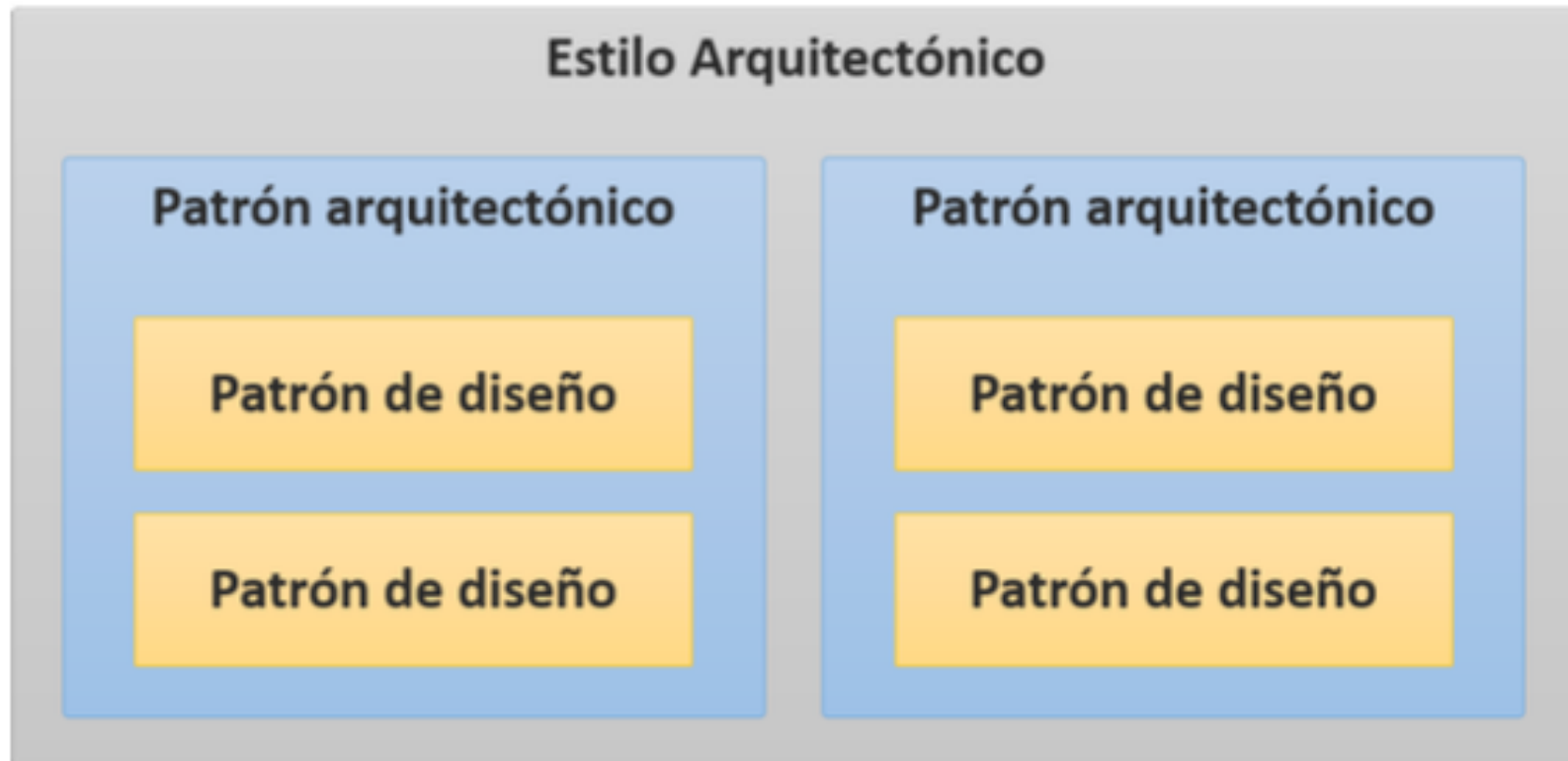


Mal

Bien

¿Cómo acertar de antemano? → ***Estilos arquitectónicos***

Recordemos que ...



... cada concepto visto opera en diferentes niveles de abstracción y se puede observar la relación que existe entre ellos de donde: un estilo arquitectónico puede ser implementado mediante 1 o más patrones arquitectónicos y estos a su vez también pueden ser implementados mediante 1 o más patrones de diseño.

Estilos Arquitectónicos (unidad I)

Se define **estilo arquitectónico** como la abstracción de distintas arquitecturas software (Klein et al., 1999).

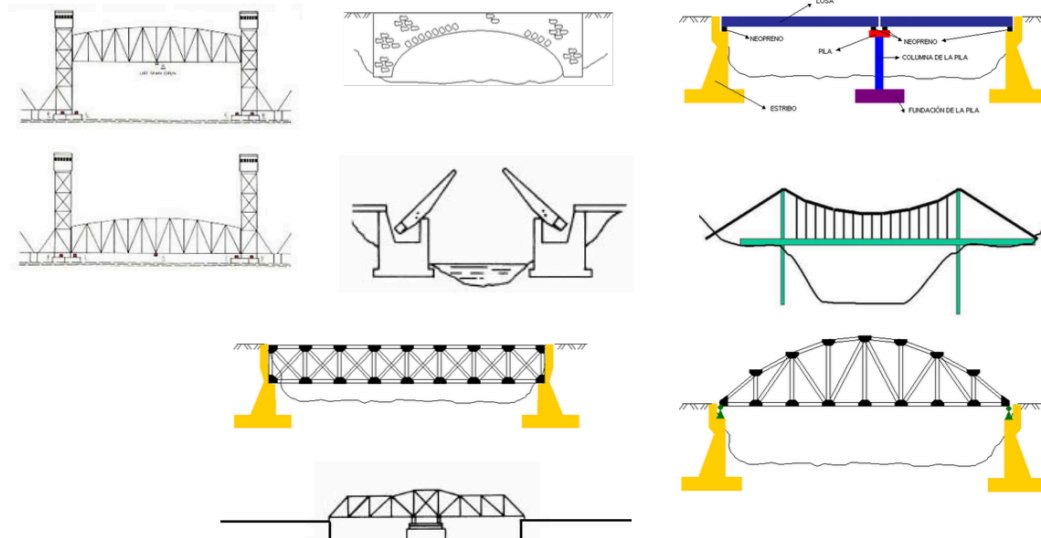
Es un conjunto de restricciones que definen un conjunto o familia de arquitecturas afines, que satisfacen dichas restricciones.

Los estilos expresan la arquitectura en el sentido más formal y teórico, constituyendo un tópico esencial de lo que Goguen ha llamado el campo “seco” de la disciplina [Goguen, 1992].

Estilos arquitectónicos de puentes

Se puede ver como un meta-modelo o macro-arquitectura.

En grandes sistemas es común combinar varios estilos arquitecturales.





Principales usos

- Sirven para sintetizar estructuras de soluciones
- Pocos estilos abstractos encapsulan una enorme variedad de configuraciones concretas
- Definen los patrones posibles de las aplicaciones
- Permiten evaluar arquitecturas alternativas con ventajas y desventajas conocidas ante diferentes conjuntos de requerimientos no funcionales

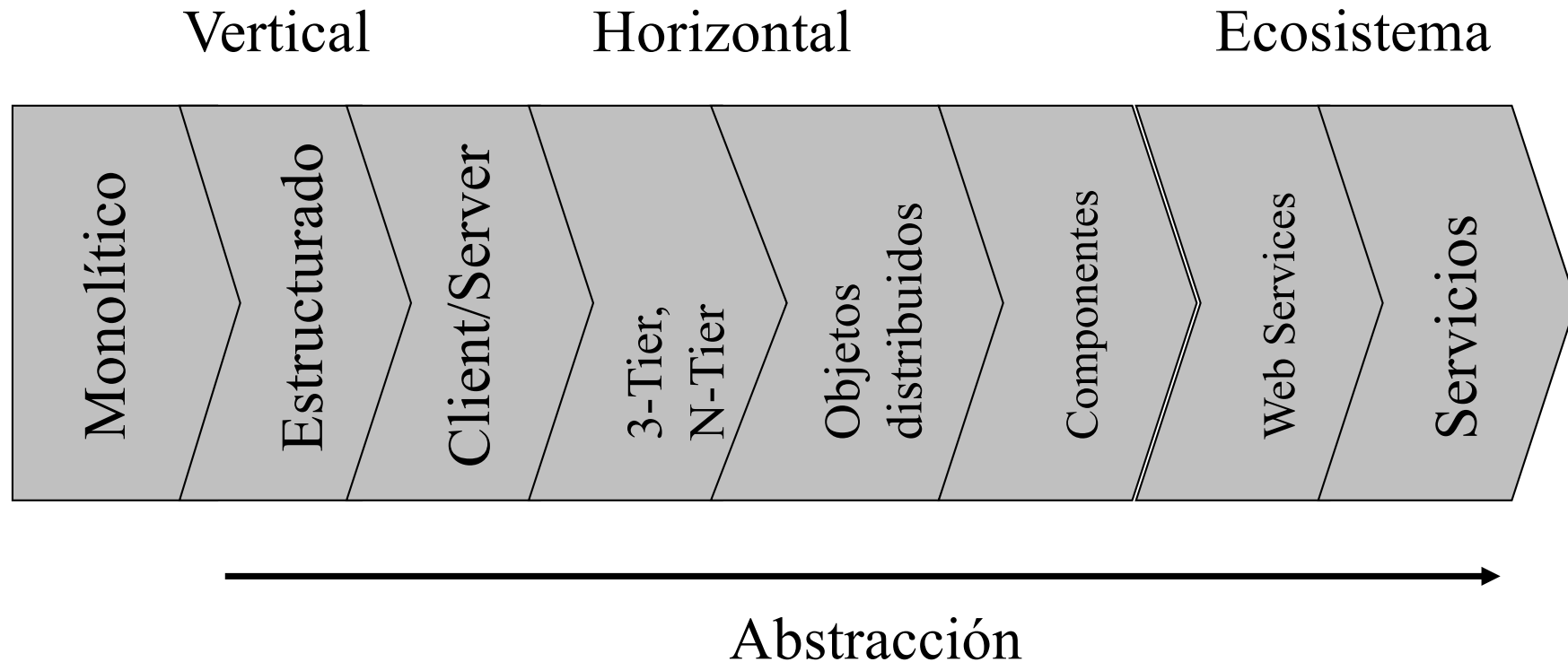


Ejemplo

- Mala práctica:
 - Aplicaciones clientes que consultan si sucedió algo
 - *Listener* de HTTP, Archivo, Colas
- Buena práctica:
 - Estilo basado en Eventos

Contexto de situación y antecedentes

- Evolución de la arquitectura:





Propiedades

	Programación Estructurada	Objetos	Componentes	Servicios
Granularidad	Muy fina	Fina	Intermedia	Gruesa
Contrato	Definido	Privado/Publico	Publico	Publicado
Reusabilidad	Baja	Baja	Intermedia	Alta
Acoplamiento	Fuerte	Fuerte	Débil	Muy débil
Dependencias	Tiempo de Compilación	Tiempo de Compilación	Tiempo de Compilación	Run-Time
Ámbito de Comunicación	Intra-Aplicación	Intra-Aplicación	Inter-Aplicaciones	Inter-Empresas



Catálogo de estilos

(... más representativos)

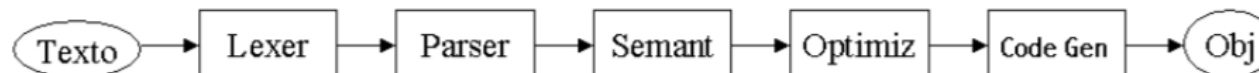
- Estilos de Flujo de Datos
 - Tubería y filtros
- Estilos Centrados en Datos
 - Arquitecturas de Pizarra o Repositorio
 - Repositorio compartido
- Estilos de Llamada y Retorno
 - Model-View-Controller (MVC)
 - Arquitecturas en Capas
 - Arquitecturas Orientadas a Objetos
 - Arquitecturas Basadas en Componentes
- Estilos de Código Móvil
 - Arquitectura de Máquinas Virtuales
- Estilos heterogéneos
 - Sistemas de control de procesos
 - Arquitecturas Basadas en Atributos
- Estilos Ciente / Servidor
 - Arquitectura Cliente / Servidor
- Estilos Peer-to-Peer
 - Arquitecturas Basadas en Eventos
 - Arquitecturas Orientadas a Servicios
 - Arquitecturas Basadas en Recursos
- Estilos de la Nube
 - ...

Estilos de Flujo de datos

Esta familia de estilos enfatiza la reutilización y la modificabilidad. Es apropiada para sistemas que implementan transformaciones de datos en pasos sucesivos

Tubería y Filtro

Una tubería (*pipeline*) es una popular arquitectura que conecta componentes computacionales (filtros) a través de conectores (*pipes*), de modo que las computaciones se ejecutan a la manera de un flujo. Los datos se transportan a través de las tuberías entre los filtros, transformando gradualmente las entradas en salidas. [...]



Estilos centrados en Datos

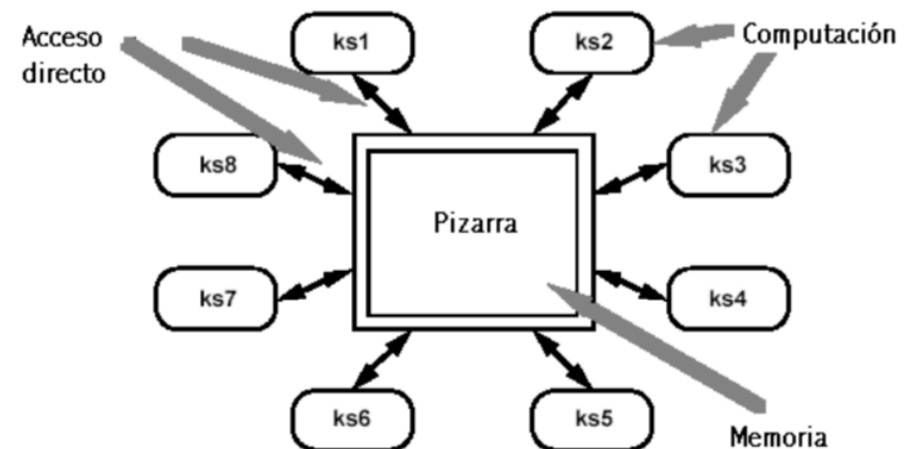
Esta familia de estilos enfatiza la integrabilidad de los datos. Se estima apropiada para sistemas que se fundan en acceso y actualización de datos en estructuras de almacenamiento.

Arquitecturas de Pizarra o Repositorio

En esta arquitectura hay dos componentes principales: una estructura de datos que representa el estado actual y una colección de componentes independientes que operan sobre él [SG96]. En base a esta distinción se han definidos dos subcategorías principales del estilo:

En base a esta distinción se han definidos dos subcategorías principales del estilo:

- Si los tipos de transacciones en el flujo de entrada definen los procesos a ejecutar, el repositorio puede ser una base de datos tradicional (implícitamente no cliente-servidor).
- Si el estado actual de la estructura de datos dispara los procesos a ejecutar, el repositorio es lo que se llama una pizarra pura o un tablero de control.





Estilos centrado en Datos

Repositorio Compartido

- Los datos comunes a los subsistemas se almacenan en una base de datos central o repositorio.
- Se emplea cuando se comparten muchos datos.
- **Ventajas**
 - Eficiente para compartir grandes cantidades de datos.
 - Los subsistemas no necesitan “preocuparse” de la gestión (backups, seguridad, ...) de los datos.
 - Existe un modelo de datos compartido (esquema del repositorio).
- **Desventajas**
 - Todos los subsistemas deben usar el mismo modelo de datos.
 - La evolución de datos es difícil y costosa.
 - No caben políticas de gestión de datos específicas.
 - Es difícil hacer una distribución eficiente.



Estilos de llamada y retorno

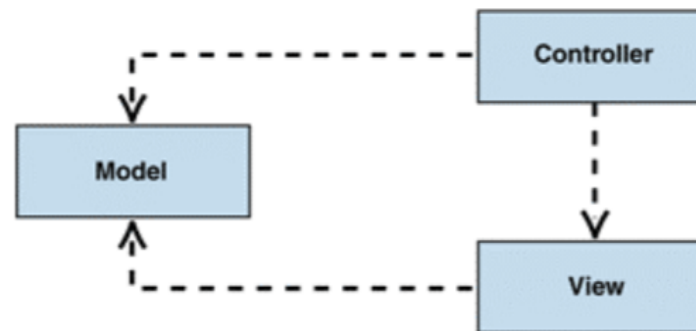
Esta familia de estilos enfatiza la modificabilidad y la escalabilidad.

Son los estilos más generalizados en sistemas en gran escala.

Miembros de la familia son las arquitecturas de programa principal y subrutina, los sistemas basados en llamadas a procedimientos remotos, los sistemas orientados a objeto y los sistemas jerárquicos en capas.

Model-View-Controller (MVC)

El patrón conocido como Modelo-Vista-Controlador (MVC) separa el modelado del dominio, la presentación y las acciones basadas en datos ingresados por el usuario en tres clases diferentes



- **Modelo.** El modelo administra el comportamiento y los datos del dominio de aplicación, responde a requerimientos de información sobre su estado (usualmente formulados desde la vista) y responde a instrucciones de cambiar el estado (habitualmente desde el controlador).
- **Vista.** Maneja la visualización de la información.
- **Controlador.** Interpreta las acciones del ratón y el teclado, informando al modelo y/o a la vista para que cambien según resulte apropiado.

Arquitectura en Capas

- Organiza el sistema en un conjunto de capas, cada una de las cuales provee una serie de servicios a las capas superiores usando los de las capas inferiores.

Sistema de Gestión de la Configuración del Software

Sistema de Gestión de Objetos

Sistema de Base de Datos

Sistema Operativo

Arquitectura Orientada a Objetos



Los componentes de este estilo son los objetos, o más bien instancias de los tipos de datos abstractos.

Podemos resumir las características en:

- Los componentes del estilo se basan en principios OO: encapsulamiento, herencia y polimorfismo. Son asimismo las unidades de modelado, diseño e implementación, y los objetos y sus interacciones son el centro de las incumbencias en el diseño de la arquitectura y en la estructura de la aplicación.
- Las interfaces están separadas de las implementaciones. En general la distribución de objetos es transparente, y en el estado de arte de la tecnología (lo mismo que para los componentes en el sentido de CBSE) apenas importa si los objetos son locales o remotos.

Arquitectura basada en Componentes



Los sistemas de software basados en componentes se basan en principios definidos por una ingeniería de software específica

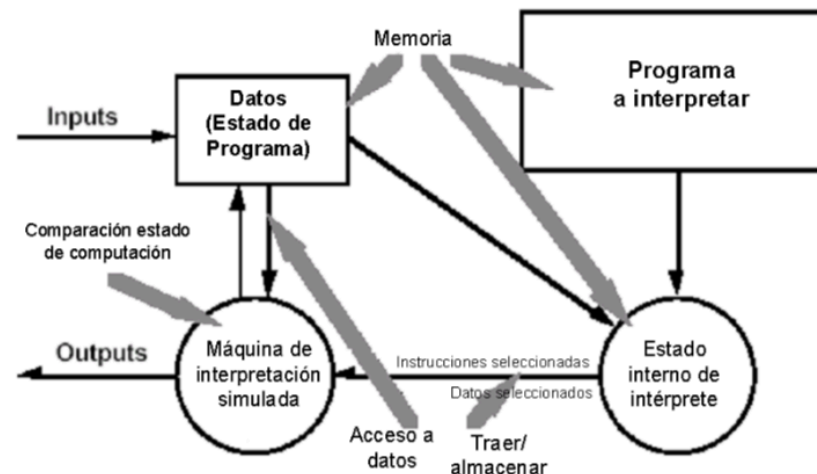
Hay un buen número de definiciones de componentes, pero Clemens Alden Szyperski proporciona una que es bastante operativa: “un componente de software, dice, es una unidad de composición con interfaces especificadas contractualmente y dependencias del contexto explícitas” [Szyperski, 2002].

Estilos de Código Móvil

Esta familia de estilos enfatiza la portabilidad. Ejemplos de la misma son los intérpretes, los sistemas basados en reglas y los procesadores de lenguaje de comando.

Arquitectura de Máquinas Virtuales

- La arquitectura de máquinas virtuales se ha llamado también intérpretes basados en tablas. De hecho, todo intérprete involucra una máquina virtual implementada en software.
- El estilo comprende básicamente dos formas o sub-estilos, que se han llamado intérpretes y sistemas basados en reglas.



Estilos heterógeneos

Incluimos en este grupo formas compuestas o indóciles a la clasificación en las categorías habituales.

Sistemas de control de procesos

Desde el punto de vista arquitectónico, caracterizan no sólo por los tipos de componentes, sino por las relaciones que mantienen entre ellos. El objetivo de un sistema de esta clase es mantener ciertos valores dentro de ciertos rangos especificados, llamados puntos fijos o valores de calibración; el caso más clásico es el de los termostatos.

Arquitecturas Basadas en Atributos

Asocia a la definición del estilo arquitectónico, un *framework* de razonamiento (ya sea cuantitativo o cualitativo) basado en modelos de atributos específicos.

Su objetivo se funda en la premisa que dicha asociación proporciona las bases para crear una disciplina de diseño arquitectónico, tornando el diseño en un proceso predecible, antes que en una metodología ad hoc.

Estilos de Control

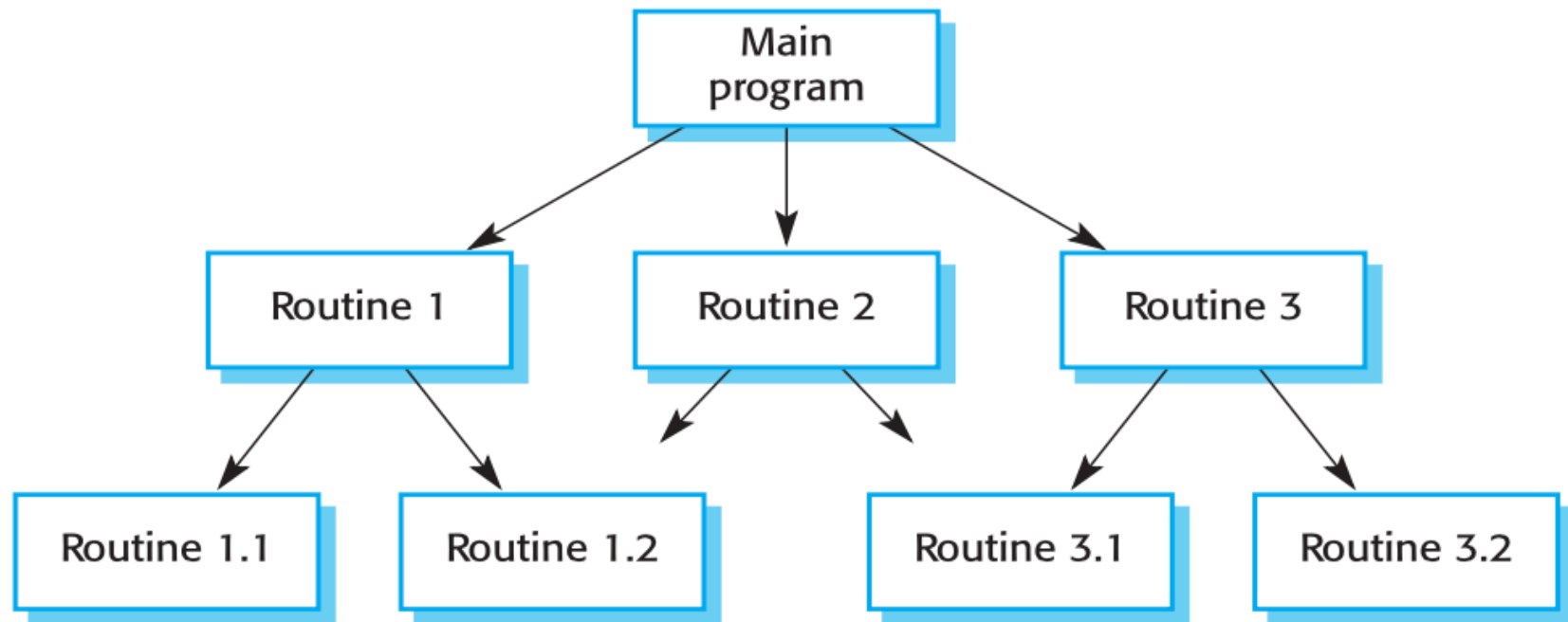
- Estos estilos arquitecturales están enfocados al **flujo de control entre subsistemas**.
- **Control Centralizado**
 - Un subsistema tiene toda la responsabilidad para controlar, iniciar y parar los otros subsistemas.
 - Llamada–Retorno (Call-Return)
 - Gestor (Manager)
- **Control basado en Eventos**
 - Cada subsistema puede responder a eventos generados externamente por los otros subsistemas o el entorno del sistema.
 - Difusión (Broadcast)
 - Guiado por Interrupciones

Estilos de Control II

- **Control Centralizado**

- **Llamada-Retorno**

- Jerarquía Top-Down de Subrutinas (operaciones)

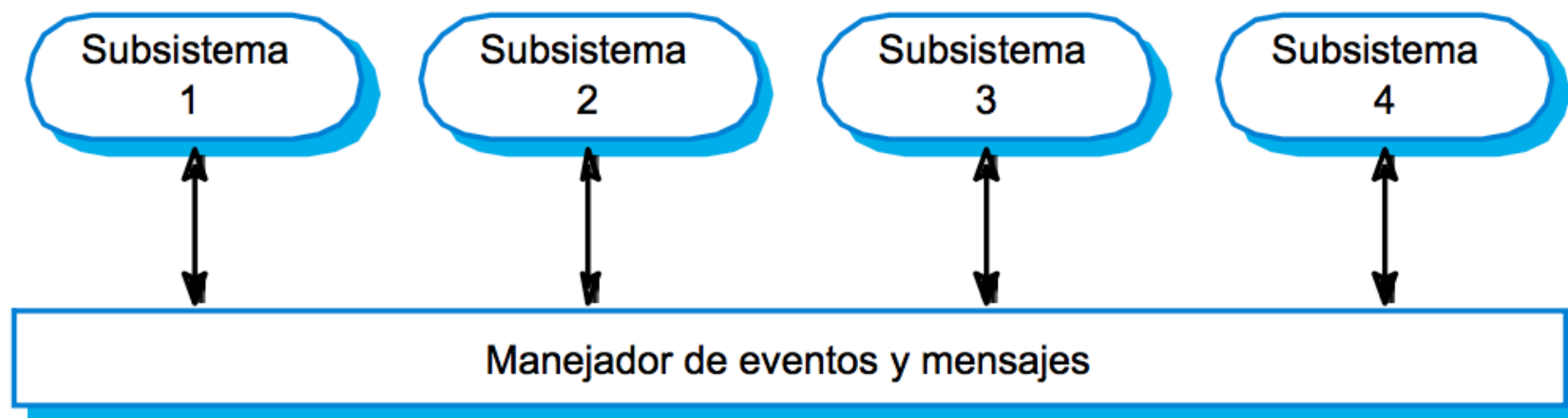


Estilos de Control III

- **Control basado en Eventos**

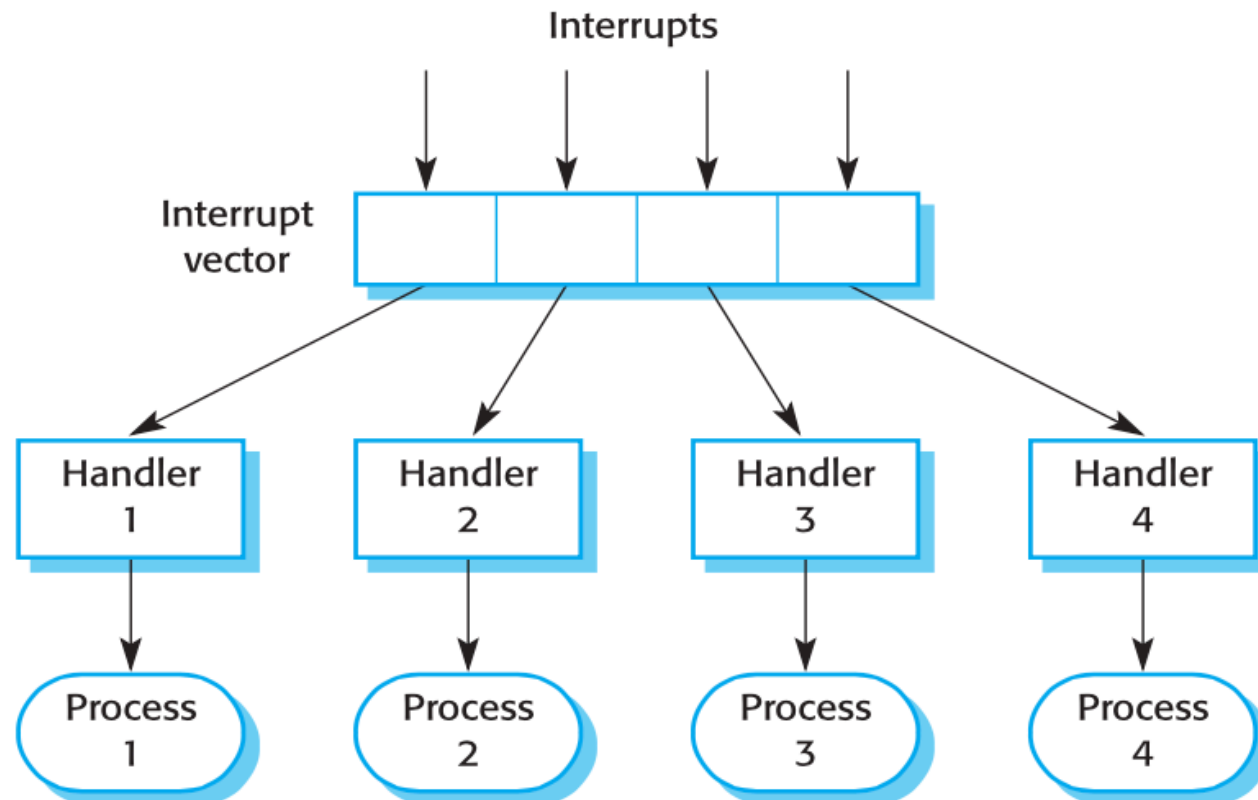
- **Difusión (Broadcast)**

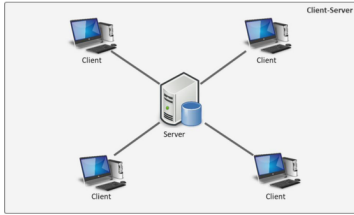
- Cuando ocurre un evento el control se transfiere al subsistema que puede tratarlo.
 - Cada subsistema decide sobre los eventos que le interesan.



Estilos de Control IV

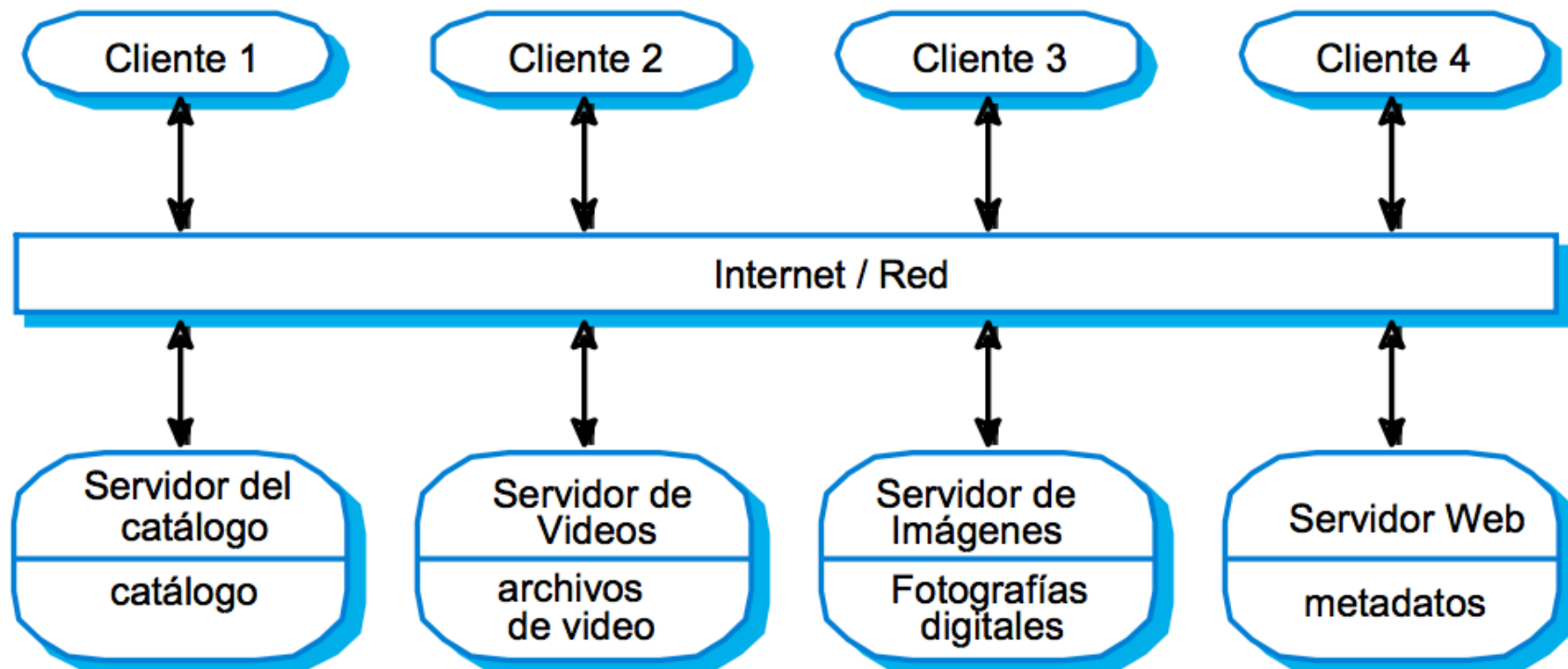
- **Control basado en Eventos**
 - **Guiado por Interrupciones**
 - Cada interrupción tiene un manejador.





Estilos Cliente /Servidor

- Estructura un sistema distribuido en:
 - **Servidores** autosuficientes que proveen servicios (impresión, gestión de datos, ..).
 - **Clientes** que invocan dichos servicios.



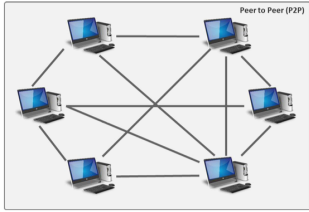
Estilos Cliente /Servidor II

■ **Ventajas**

- La distribución de datos es real y directa;
- Se hace uso eficaz de sistemas en red. El hardware para ello puede ser barato;
- Es fácil añadir nuevos servidores o actualizar los existentes.

■ **Desventajas**

- Los subsistemas usan diferentes modelos de datos. Esto puede hacer ineficiente el intercambio de datos;
- Gestión redundante en cada servidor;
- No existe un registro central de nombres y servicios. Puede ser difícil averiguar qué servidores y servicios están disponibles.



Estilos Peer-to-Peer

Esta familia, también llamada de componentes independientes, enfatiza la modificabilidad por medio de la separación de las diversas partes que intervienen en la computación.

Consiste por lo general en procesos independientes o entidades que se comunican a través de mensajes. Cada entidad puede enviar mensajes a otras entidades, pero no controlarlas directamente.

Arquitecturas Basadas en Eventos

Desde el punto de vista arquitectónico, los componentes de un estilo de invocación implícita son módulos cuyas interfaces proporcionan tanto una colección de procedimientos como un conjunto de eventos. Los procedimientos se pueden invocar a la manera usual en modelos orientados a objeto, o mediante el sistema de suscripción del patrón de diseño observer. Ej: C2 o Chiron 2 (comunicación por bus con eventos).

Arquitecturas Orientada a Servicios

Un Web service es un sistema de software diseñado para soportar interacción máquina-a-máquina sobre una red. Posee una interfaz descrita en un formato procesable por máquina (específicamente WSDL).

Lo que hace diferentes a los Web services de otros mecanismos de RPC como RMI, CORBA o DCOM es que utiliza estándares de Web para los formatos de datos y los protocolos de aplicación. Esto permite que las aplicaciones interoperen con mayor libertad,

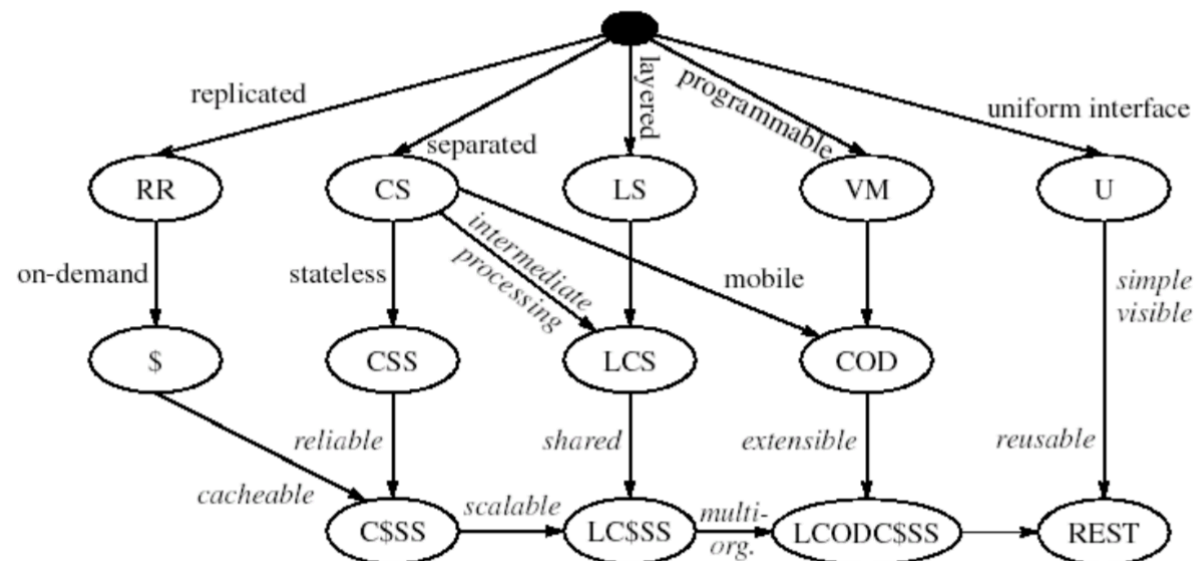
Estilos Peer-to-Peer II

Arquitecturas basadas en Recursos

En una síntesis muy apretada, podría decirse que REST (REpresentational State Transfer) define recursos identificables y métodos para acceder y manipular el estado de esos recursos.

La polémica sobre si REST constituye una revolucionaria inflexión en los estilos orientados a servicios (“la nueva generación de Web services”) o si es una variante colateral todavía subsiste.

El lema de REST es, después de todo, “Adiós objetos, adiós RPC”.





Estilos de la “nube”

Estilo de arquitectura	Administración de dependencias	Tipo de dominio
N niveles	Niveles horizontales divididos por subred	Dominio empresarial tradicional. La frecuencia de las actualizaciones es baja.
Web-Cola-Trabajo	Trabajos de front-end y back-end, desacoplados mediante mensajería asincrónica.	Dominio relativamente sencillo con algunas tareas de uso intensivo de recursos.
Microservicios	Servicios descompuestos verticalmente (funcionalmente) que se llaman mutuamente mediante API.	Dominio complicado. Actualizaciones frecuentes.
Arquitectura basada en eventos.	Productores y consumidores. Vista independiente por cada subsistema.	IoT y sistemas en tiempo real
Macrodatos	Divide un conjunto de datos grande en fragmentos pequeños. Procesamiento en paralelo en los conjuntos de datos locales.	Análisis de datos por lotes y en tiempo real. Análisis predictivo mediante Machine Learning.
Big Compute	Asignación de datos a miles de núcleos.	Dominios con procesos intensivos como simulaciones.

Estilos arquitectónicos y atributos de calidad



Estilo	Descripción	Atributos asociados	Atributos en conflicto
<i>Datos Centralizados</i>	Sistemas en los cuales cierto número de clientes accede y actualiza datos compartidos de un repositorio de manera frecuente.	Integrabilidad Escalabilidad Modificabilidad	Desempeño
<i>Flujo de Datos</i>	El sistema es visto como una serie de transformaciones sobre piezas sucesivas de datos de entrada. El dato ingresa en el sistema, y fluye entre los componentes, de uno en uno, hasta que se le asigne un destino final (salida o repositorio).	Reusabilidad Modificabilidad Mantenibilidad	Desempeño
<i>Máquinas Virtuales</i>	Simulan alguna funcionalidad que no es nativa al hardware o software sobre el que está implementado.	Portabilidad	Desempeño
<i>Llamada y Retorno</i>	El sistema se constituye de un programa principal que tiene el control del sistema y varios subprogramas que se comunican con éste mediante el uso de llamadas.	Modificabilidad Escalabilidad Desempeño	Mantenibilidad Desempeño
<i>Componentes Independientes</i>	Consiste en un número de procesos u objetos independientes que se comunican a través de mensajes.	Modificabilidad Escalabilidad	Desempeño Integrabilidad



Bibliografía

Software Architecture. Perspective on an Emerging Discipline. Mary Shaw, David Garlan. Prentice Hall 1996.

Software Architecture in Practice (SEI Series in Software Engineering). Bass, L., Clements, P., Kasman, R., Bass, K. 3ra. ed. Addison-Wesley Pub Co; 2012.

Documenting Software Architectures: Views and Beyond. Clements, P. et al. 2da. ed. Addison-Wesley, 2010.

Evaluating Software Architectures: Methods and Case Studies. Clements, P. et al. 1ra. ed. Addison-Wesley.

Pattern-Oriented Software Architecture. A System of Patterns. F. Buschman et al. New York, John Wiley and Sons, 1996.

Pattern-Oriented Software Architecture. Patterns for Concurrent and Networked Objects. Volumen 2. Douglas Schmith, Michael Stal, Hans Rohnert, Frank Buschman. John Wiley and Sons, 2000.

IEEE 1471 Recommended Practice for Architectural Description.

<https://standards.ieee.org/findstds/standard/1471-2000.html>

The Worldwide Institute of Software Architects. <http://www.wwisa.org/wwisamain/index.htm>

International Association of Software Architects. <http://iasaglobal.org/>

Unified Modeling Language (UML). <http://www.uml.org/>

Software Architecture Links. <http://www.bredemeyer.com/links.htm>

Martin Fowler. GUI Architecture patterns. <http://martinfowler.com/eaaDev/uiArchs.html>



UNIVERSIDAD CHAMPAGNAT



www.uch.edu.ar



[uch.argentina](https://www.facebook.com/uch.argentina)



[@uchampagnat](https://twitter.com/uchampagnat)

CAMPUS CENTRAL

Belgrano 721 - Godoy Cruz, Mendoza - C.P.5501 - Argentina.
Teléfono 0261-4248443

CAMPUS SAN RAFAEL

Libertador 150 - San Rafael - Mendoza - C.P.5600 - Argentina.
Teléfono 0260-4424812

CAMPUS VALLE DE UCO

Pueyrredón 58, 1º Piso - Tunuyán - Mendoza - C.P.5560 - Argentina
Teléfono 02622-460644